

Universidade Estadual de Campinas
Instituto de Computação

Tarefa 6

Persistência de Objetos e Testes Automatizados com JUnit



MC322 - Programação Orientada a Objetos

Professor: Marcelo da Silva Reis
PEDs: Athyrson Ribeiro, Matheus Lozano
PADs: João Pedro Frare

---

1 Descrição Geral

Ao longo das tarefas anteriores, foi desenvolvida uma planta industrial utilizando conceitos de orientação a objetos, como encapsulamento, herança, polimorfismo, interfaces, padrões de projeto, composição e tratamento de erros. O projeto também passou a utilizar Gradle, documentação Javadoc e uma estrutura mais organizada de classes.

Entretanto, até o momento, as informações mantidas pelos objetos podem existir apenas durante a execução do programa. Ao encerrar a aplicação, o estado da planta é perdido, exigindo que as informações sejam cadastradas novamente na próxima execução. Além disso, à medida que o sistema se torna mais complexo, executar manualmente todas as funcionalidades a cada alteração deixa de ser uma maneira eficiente de verificar seu funcionamento.

Nesta tarefa, a dupla deverá implementar mecanismos de **persistência de dados**, permitindo salvar e recuperar informações da planta industrial, desenvolver uma suíte de **testes automatizados com JUnit** e adotar uma **organização estruturada de pastas e pacotes em Java**.

1.1 Evolução da Planta

Ao final desta tarefa, a planta deverá:

- Salvar em arquivos informações relevantes sobre seu estado;
- Recuperar essas informações em uma execução posterior;
- Manter a consistência dos objetos após a recuperação dos dados;
- Tratar situações de falha durante a leitura e a escrita dos arquivos;
- Separar adequadamente as responsabilidades de persistência das regras de negócio;
- Possuir testes automatizados utilizando JUnit;
- Executar os testes por meio do Gradle;
- Adotar uma organização convencional de projetos Java com Gradle, utilizando pacotes coesos e separação de responsabilidades;
- Preservar as funcionalidades desenvolvidas nas tarefas anteriores.

2 Objetivos

- Compreender a diferença entre o estado em memória e os dados persistidos;
- Implementar operações de salvamento e recuperação de informações;
- Utilizar XML para representar dados persistidos e reconstruir objetos a partir dessa representação;
- Compreender o papel dos testes unitários na manutenção de sistemas orientados a objetos;
- Escrever testes utilizando JUnit;

- Verificar tanto comportamentos esperados quanto situações de erro;
- Estruturar o código-fonte aplicando organização convencional de projetos Java com Gradle, utilizando pacotes coesos e separação de responsabilidades;
- Manter a consistência entre a implementação, a persistência e os testes.

3 Atividades

3.1 Implementação da Persistência em XML

A dupla deverá implementar mecanismos para **salvar e carregar os dados da planta industrial utilizando arquivos XML**. A implementação deverá contemplar obrigatoriamente um conjunto mínimo de dados para recuperar um estado útil e comparável da planta, englobando pelo menos:

- o estoque atual de matérias-primas;
- o estado e a configuração das máquinas e de seus subcomponentes (composição);
- as demandas de produção e seus respectivos estados (`StatusDemanda`).

Outros elementos complementares (como o orçamento atual ou produtos no armazém) também poderão ser persistidos a critério da dupla.

A implementação deve permitir:

1. **Salvar:** registrar em um arquivo XML os dados relevantes da planta.
2. **Carregar:** ler um arquivo XML e reconstruir os objetos correspondentes.
3. **Atualizar:** salvar novamente as informações após alterações relevantes.
4. **Validar:** verificar se os dados lidos possuem estrutura e valores compatíveis com as regras do domínio.

Não será suficiente gerar um arquivo XML destinado apenas à leitura humana. Os dados devem ser lidos pela aplicação e utilizados para reconstruir o estado do sistema.

3.1.1 Estrutura do XML

O arquivo deverá possuir uma estrutura hierárquica coerente com o domínio, utilizando elementos e atributos com nomes significativos. Por exemplo, uma representação simplificada de matérias-primas poderia ser:

```
<?xml version="1.0" encoding="UTF-8"?>
<planta>
    <materiasPrimas>
        <materiaPrima id="MP-01">
            <nome>Aco</nome>
            <quantidade>150.0</quantidade>
            <unidade>kg</unidade>
        </materiaPrima>
        <materiaPrima id="MP-02">
            <nome>Plastico</nome>
            <quantidade>80.0</quantidade>
            <unidade>kg</unidade>
        </materiaPrima>
    </materiasPrimas>
</planta>
```

O exemplo é ilustrativo. A dupla deverá definir uma estrutura compatível com as classes e os relacionamentos efetivamente implementados no projeto. Lembrem-se de adequar os nomes dos campos à temática escolhida para a fábrica.

3.1.2 Organização das Responsabilidades

A persistência deverá ser implementada de maneira compatível com a arquitetura desenvolvida nas tarefas anteriores. As entidades de domínio, como `Maquina`, `Produto` e `MateriaPrima` não deverão concentrar toda a lógica de leitura e escrita de arquivos. Recomenda-se criar uma classe responsável pelo armazenamento e pela recuperação dos dados, por exemplo:

```java
public class PersistenciaPlanta {
    public void salvar(Planta planta) {
        ...
    }

    public Planta carregar() {
        ...
    }
}
```

Listing 1: Exemplo Genérico de Classe de Persistência

A assinatura acima é apenas ilustrativa; a solução deverá ser adaptada às classes e à organização efetivamente utilizada no projeto. Os nomes e a quantidade de classes ficam a critério da dupla, desde que as responsabilidades estejam bem definidas e sejam adequados ao tema da fábrica.

A dupla deverá evitar duplicar a lógica de leitura e escrita em diferentes partes do sistema.

3.1.3 Integração com a Aplicação

A aplicação deverá oferecer uma maneira de salvar e carregar o estado da planta. A dupla deverá adaptar o menu principal, acrescentando opções como:

```
1. Consultar planta
2. Executar produção
3. Consultar estoque
4. Salvar estado da planta
5. Carregar estado da planta
0. Encerrar
```

O menu acima é apenas um exemplo. As opções deverão ser integradas às funcionalidades já existentes.

A aplicação também deverá definir como lidar com o carregamento inicial. A ausência do arquivo na primeira execução poderá ser tratada como uma situação normal ou algo que gere falha, desde que esse comportamento esteja definido e justificado pela dupla. Por exemplo, poderá tentar recuperar os dados previamente salvos e, quando não houver um arquivo de persistência, inicializar uma planta com os dados padrão do projeto.

Os dados salvos em arquivo deverão permanecer disponíveis após o encerramento da aplicação, permitindo que sejam recuperados em execuções posteriores. A dupla deverá garantir que o encerramento do programa não apague nem invalide os dados previamente salvos.

Quando um carregamento for concluído com sucesso, a aplicação deverá substituir o estado atual da planta pelo estado recuperado do arquivo, evitando duplicações ou combinações indevidas entre os dados carregados e os dados existentes.

3.1.4 Tratamento de Erros de Persistência

A leitura e a escrita de arquivos estão sujeitas a falhas. A dupla deverá tratar adequadamente essas situações, utilizando os mecanismos de tratamento de exceções desenvolvidos na Tarefa 4.

Deverão ser consideradas, no mínimo, as seguintes situações:

- arquivo de persistência inexistente;
- arquivo sem conteúdo, quando isso não for permitido;
- arquivo com formato inválido ou dados corrompidos;
- falha de leitura ou escrita;
- dados incompatíveis com as regras do domínio.

A aplicação deverá apresentar mensagens compreensíveis e evitar que uma falha de persistência deixe o sistema em um estado inconsistente. O carregamento deverá validar os dados antes de alterar o estado atual da planta. Se ocorrer uma falha, o estado válido previamente existente deverá ser preservado.

3.2 Implementação dos Testes Unitários

Devem ser desenvolvidos testes unitários utilizando JUnit para verificar o comportamento das principais classes e operações do sistema. Os testes deverão avaliar o comportamento observável das classes, sem depender desnecessariamente de detalhes internos de implementação.

Os casos de teste deverão incluir as seguintes categorias:

Comportamento das classes de domínio: Implementar testes para operações relevantes do sistema, como:

- criação e configuração de objetos;
- alteração de estado;
- aplicação das regras de negócio;
- execução de operações de produção.

A dupla deverá escolher os casos mais relevantes para seu domínio. Não é necessário testar indiscriminadamente todos os getters e setters.

Persistência e recuperação: Implementar testes que verifiquem:

- salvamento de dados;
- carregamento de dados previamente salvos;
- preservação das informações após salvar e carregar;
- recuperação de coleções e relacionamentos relevantes.

Os testes deverão demonstrar que as informações recuperadas correspondem às informações originalmente armazenadas.

Situações de erro: Implementar testes para verificar o comportamento esperado em situações inválidas, como:

- dados de entrada inválidos;
- tentativa de executar uma operação que viole uma regra de negócio (ex.: estoque insuficiente);
- arquivo inexistente, quando aplicável;
- arquivo corrompido ou incompatível;
- falha que deva resultar em uma exceção específica.

A dupla deverá escolher situações de erro que façam sentido para a implementação e verificar se o sistema reage conforme o comportamento definido.

3.2.1 Qualidade e Isolamento dos Testes

Os testes deverão ser executáveis de maneira independente e repetível. Para isso:

- cada teste deve preparar os dados de que necessita;
- os resultados não devem depender da ordem de execução dos testes;
- o estado criado por um teste não pode interferir nos demais;
- os testes não devem depender de uma execução manual anterior da aplicação;
- os arquivos utilizados pelos testes de persistência devem ser isolados dos arquivos de dados reais da aplicação.

Recomenda-se utilizar arquivos temporários e recursos do JUnit para preparar e limpar os dados de teste. Além disso, os testes não deverão exigir interação manual com o terminal, entrada pelo teclado ou intervenção do usuário. Quando uma operação produzir uma exceção, o teste deverá verificar se ela corresponde ao comportamento esperado, em vez de simplesmente falhar sem explicar a condição avaliada.

3.3 Documentação do Projeto via `README.md`

O projeto deverá contar com um arquivo `README.md` detalhado e bem estruturado na raiz do repositório. Além das informações gerais da aplicação, o `README.md` deve conter obrigatoriamente a seção **Escopo e Limitações**, que descreve explicitamente:

- **Dados Persistidos vs. Não Persistidos:** Explicação fundamentada dos dados obrigatórios e opcionais que a dupla escolheu tornar persistentes (salvos em XML) e justificativa clara dos dados que foram mantidos apenas em memória e não foram persistidos;
- **Explicação da Suíte de Testes:** Descrição detalhada da estratégia adotada para os testes automatizados com JUnit, explicando explicitamente o que cada classe ou grupo de testes avalia (regras de negócio, persistência, casos de exceção).

3.4 Organização Estruturada de Pastas e Pacotes (Padrão Java)

Para garantir a manutenibilidade, coesão e separação de responsabilidades do projeto, a estrutura de diretórios e pacotes em Java deverá seguir uma organização convencional de projetos Java com Gradle, utilizando pacotes coesos e separação de responsabilidades, evitando classes aglomeradas no pacote raiz (*default package*).

3.4.1 Exemplo de Organização Padrão de Projeto

Espera-se que o código-fonte seja distribuído em pacotes coesos dentro da estrutura convencional do Gradle, conforme o exemplo demonstrado abaixo:

```
projeto/
├── src/
│   ├── main/
│   │   └── java/
│   │       └── br/
│   │           └── unicamp/
│   │               └── ic/
│   │                   └── mc322/
│   │                       ├── model/       (Classes de dominio, entidades e componentes)
│   │                       ├── service/     (Regras de negocio e gerenciadores de producao)
│   │                       ├── strategy/    (Implementacoes do padrao Strategy)
│   │                       ├── persistence/ (Modulos de leitura e escrita XML)
│   │                       ├── exception/   (Excecoes de dominio personalizadas)
│   │                       └── Main.java    (Classe principal de execucao)
│   └── test/
│       └── java/
│           └── br/
│               └── unicamp/
│                   └── ic/
│                       └── mc322/           (Classes de teste JUnit 5)
├── build.gradle
├── settings.gradle
└── gradlew
```

A organização exata dos subpacotes pode ser adaptada pela dupla conforme a arquitetura adotada, desde que mantenha a separação lógica clara entre domínio, persistência, regras de negócio e testes.

3.4.2 Configuração do JUnit no Gradle

O projeto deverá utilizar **JUnit 5** para a implementação dos testes automatizados.

O arquivo build.gradle deverá incluir as dependências e configurações necessárias para utilizar o JUnit 5. A configuração deverá permitir executar os testes com:

```
./gradlew test
```

O comando `build` deverá executar os testes como parte do processo de construção.

3.4.3 Relatório de Execução dos Testes

A dupla deverá executar os testes utilizando o Gradle e verificar os resultados. O projeto deverá apresentar:

- testes identificados pelo JUnit;
- execução automatizada pelo Gradle;
- indicação dos testes aprovados e reprovados;
- relatório de execução gerado pelo processo de testes.

O relatório HTML normalmente estará disponível em:

```
build/reports/tests/test/index.html
```

A localização deverá ser confirmada conforme a configuração do projeto.

Todos os testes entregues deverão estar aprovados na versão final da release. Testes que falham intencionalmente não deverão ser utilizados como substitutos para testes que verifiquem corretamente os comportamentos esperados.

4 Integração com as Tarefas Anteriores

A dupla deverá garantir que:

- as classes e regras de negócio existentes continuem funcionando;
- os padrões de projeto e as interfaces previamente introduzidos sejam preservados;
- a composição e os relacionamentos da Tarefa 5 sejam respeitados durante a recuperação dos objetos;
- a documentação Javadoc continue disponível, e as novas classes também sejam documentadas;
- o tratamento de exceções seja aplicado também à persistência;
- o projeto continue compilando e executando pelo Gradle;
- os testes possam ser executados sem iniciar a aplicação interativa.

5 Avaliação

A avaliação levará em conta os seguintes critérios:

1. **Persistência e Recuperação de Dados (2,5 pts)**
   - Implementação de salvamento e carregamento contemplando obrigatoriamente o mínimo exigido (estoque, máquinas/componentes e demandas);
   - Preservação dos dados relevantes entre execuções;
   - Recuperação consistente dos relacionamentos entre objetos;
   - Integração com as funcionalidades existentes.
2. **Testes Unitários com JUnit (2,5 pts)**
   - Cobertura de regras de negócio relevantes;
   - Testes de persistência e recuperação;
   - Testes de situações de erro;
   - Utilização adequada das asserções e dos recursos do JUnit;
   - Testes independentes, repetíveis e sem interação manual.
3. **Organização e Estrutura de Pacotes (2,0 pts)**
   - Aplicação correta de organização convencional de projetos Java com Gradle, utilizando pacotes coesos e separação de responsabilidades;
   - Separação clara de responsabilidades arquiteturais (modelo, persistência, serviços, exceções);

- Ausência de classes aglomeradas no pacote raiz.

4. **Organização e Tratamento de Erros (2,0 pts)**
   - Separação adequada entre domínio e persistência;
   - Organização e reutilização do código;
   - Validação dos dados recuperados;
   - Tratamento de falhas de leitura e escrita;
   - Preservação da consistência do sistema diante de erros.
5. **Integração com Gradle e Relatório (1,0 pt)**
   - Configuração correta do JUnit 5;
   - Execução dos testes por `./gradlew test`;
   - Execução dos testes durante `./gradlew build`;
   - Geração do relatório de testes;
   - Presença da seção **Escopo e Limitações** no `README.md` contendo a justificativa dos dados e a explicação dos testes;
   - Todos os casos de teste passando.
6. **Penalidade:** Não compilação via Gradle = -5,0 pontos.

6 Entrega

1. **GitHub:** Release denominada `tarefa6-RA1-RA2` contendo o código fonte atualizado com pacotes coesos, arquivo `README.md` atualizado com a seção de escopo e limitações e testes automatizados com JUnit. Os dados gerados durante a execução normal da aplicação não precisam ser incluídos na release, desde que o projeto consiga inicializar corretamente quando esses arquivos ainda não existirem.;
2. **Google Forms:** Formulário preenchido com nomes, RAs e link direto da release no GitHub.

Atenção: Crie a release primeiro, copie o link e, em seguida, submeta no Forms.
O prazo final de submissão é o dia **09/11/2026 às 23h59**. Entregas atrasadas sofrerão penalização de **25% sobre a nota obtida**. Não serão aceitas entregas efetuadas após **16/11/2026**.

6.1 Compilação e Execução

O projeto deve continuar compilando e executando utilizando a estrutura do Gradle:

```
./gradlew build
./gradlew run
./gradlew test    # para executar os testes automáticos
```

A aplicação deverá funcionar corretamente mesmo quando ainda não existir um arquivo de persistência, conforme o comportamento inicial definido pela dupla.

7 Dicas de Boas Práticas

- **Persistir não é apenas imprimir:** os dados devem poder ser recuperados pela aplicação e utilizados novamente.
- **Separe responsabilidades:** as entidades de domínio não precisam conhecer os detalhes de leitura e escrita dos arquivos.
- **Preserve as regras de negócio:** os objetos recuperados devem respeitar as mesmas invariantes dos objetos criados normalmente.
- **Cuidado com os relacionamentos:** salvar e carregar objetos relacionados exige atenção para evitar referências duplicadas ou inconsistentes.
- **Teste comportamentos, não detalhes internos:** priorize o que uma classe deve fazer, incluindo os resultados e as exceções esperados.
- **Teste casos inválidos:** situações de erro são parte importante do comportamento do sistema.
- **Isole os testes:** não utilize os arquivos reais da planta como área de trabalho dos testes automatizados.
- **Automatize a verificação:** os testes devem poder ser executados sem interação manual por meio do Gradle.
- **KISS:** implemente a persistência necessária para o domínio, sem criar uma infraestrutura desproporcional ao tamanho do projeto.
- **Pacotes coesos:** Evite colocar arquivos ‘.java‘ soltos na raiz de ‘src/main/java‘. Utilize pacotes bem definidos que reflitam a arquitetura do sistema.